Project Number Gessingh Game Report

By: Logan Bauch

1. Project Description:

   For this project, out of all the options, I decided to choose the Number Guessing Game. I made a game that picks a random number depending on the difficulty you select, and the user tries to guess it. After every time you guess a number, the program tells you if you have too high a guess, or too low a guess. You have a specific number of attempts depending on what difficulty you pick (easy, medium, or hard). All of these different difficulties are in different ranges of numbers. Easy being numbers 1-50, medium being 1-100, and hard being 1-200. Each of these has a different number of attempts: 10 for easy, 7 for medium, and 5 for hard. This part of the project was unique. I was asked to add something else to the game I was given, and I thought changing the odds (the range) and the number of guesses would make it more personal for the person playing the game. At the end of the game, it asks if the user wants to play again or not, so a player can play the game as many times as they want.

2. Approach/Logic

   I split the code into two different functions: play_game() and the main() function. The main function just shows the difficulty prompt that asks the user what difficulty they want to play (easy, medium, or hard). It also takes the input of what difficulty the user picks, and then calls the playgame function. The main function also asks the player if they want to play the game again. If the player says yes, it loops back to the start with the difficulty prompt; if the player says no (setting playing to false), it ends the while loop.

   The first thing the play_game function does is check what difficulty it was at, so it can get the number range/amount of guesses the user has for that round of the game. Then it creates a secret number using the random.randint(). 1 being the minimum, and the max range being the max; then it goes into the while loop (while attempts < max_attempts:). While the number of attempts is less than the max_attempts allowed, it keeps going, over and over, checking to see if the guess the user made was inside the range, if the guess was true/false. Also makes sure the data is valid, making sure the input is a whole number, and ends the loop when the user is out of attempts, and tells the user the “secret” number.

3. Data Structures
   I used two different dictionaries. The first is called levels, and it's a dictionary that stores the settings for each difficulty. Meaning the number range, and the maximum number of guesses the user has. The key is the difficulty name, and the value is another dictionary with those two different settings (range, max amount, attempts, and allowed attempts). The second dictionary is

called options, which connects the numbers 1, 2, and 3 to the difficulty names easy, medium, and hard. This makes it so it can be easily used in the play_game() function.

4. Challenges

The hardest part of this project for me was getting the input from the user to work right, without the program just crashing when someone types in something I didn't want. So, because of this, I had to add try and except blocks to catch these errors when someone types an extra letter, or anything that basically wasn't a number. This part of the project was by far the longest to figure out, mainly because I had to set guess to None even before the try block. This was so guess always had a value in place, regardless of whether the player typed something incorrect/invalid. Without this, the program would keep crashing.

Also, it was very annoying to figure out the case; if given bad input, it doesn't count as one of the player's guesses. So I used a valid guess, it was a variable that starts as false every time, this is so it can go through the loop and only gets set to true if the input passes all the checks. Let's say a user types a letter or a number out of the given range; it just prompts again without taking away one of the user's guesses. Also, getting the replay loop to work was something I had to trial and error through. First, I tried having everything in another while loop inside the first one to handle the replay part, but this caused it to keep asking “playing again” over and over, and sending it back to the prompt with the difficulties listed. So I ended up using a variable called playing. This variable started as True and only got set to False when the players said no to playing again said not to play again. This worked, so when playing gets set to False, the while loop notices and it just stops running completely on its own.

Possible Improvements:

If I had unlimited time for this project I would add a score tracker. This would make the game much more interactive because users could try to beat their best attempt at the game for each of the different difficulties. Also, I could have made it more customizable, maybe allowing users to pick the number of guesses they wanted and the number of attempts they wanted. I could also have added something that showed the statistics. Let's say a user picks easy difficulty, it shows the odds of them getting the number, and the continued odds when they have more and more guesses. By odds, I mean just telling them the actual percent chance they have of guessing it. I also could have made a simple change by making the yes/no prompt at the end more specific, only accepting yes and no, and ending the prompt if I entered, for example, “o”.